

```

72.     self.moves list. """
73.     if self.moves == []:
74.         return # No states in self.moves, so do nothing.
75.
76.     self.moves.pop() # Remove the current state.
77.
78.     if self.moves == []:
79.         self.resetGrid()
80.     else:
81.         # set the grid to the last move.
82.         self.grid = copy.copy(self.moves[-1])
83.
84. def display(self):
85.     """Display the current state of the grid on the screen."""
86.     print('  A B C   D E F   G H I') # Display column labels.
87.     for y in range(GRID_LENGTH):
88.         for x in range(GRID_LENGTH):
89.             if x == 0:
90.                 # Display row label:
91.                 print(str(y + 1) + ' ', end='')
92.
93.                 print(self.grid[(x, y)] + ' ', end='')
94.                 if x == 2 or x == 5:
95.                     # Display a vertical line:
96.                     print('| ', end='')
97.             print() # Print a newline.
98.
99.             if y == 2 or y == 5:
100.                 # Display a horizontal line:
101.                 print('  -----+-----+-----')
102.
103. def _isCompleteSetOfNumbers(self, numbers):
104.     """Return True if numbers contains the digits 1 through 9."""
105.     return sorted(numbers) == list('123456789')
106.
107. def isSolved(self):
108.     """Returns True if the current grid is in a solved state."""
109.     # Check each row:
110.     for row in range(GRID_LENGTH):
111.         rowNumbers = []
112.         for x in range(GRID_LENGTH):
113.             number = self.grid[(x, row)]
114.             rowNumbers.append(number)
115.         if not self._isCompleteSetOfNumbers(rowNumbers):
116.             return False
117.
118.     # Check each column:
119.     for column in range(GRID_LENGTH):
120.         columnNumbers = []
121.         for y in range(GRID_LENGTH):
122.             number = self.grid[(column, y)]
123.             columnNumbers.append(number)
124.         if not self._isCompleteSetOfNumbers(columnNumbers):
125.             return False
126.

```